بسم الله الرحمن الرحیم

پروژه میانترم سیستم عامل

اعضای گروه:
کوثر نجاری ، فاطمه رجب پور

لینک گیت هاب پروژه:

https://github.com/KNajjary/OperatingSystem-IPFS-Project

می توانید آخرین نسخه این سند را در لینک زیر مشاهده کنید:
https://docs.google.com/document/d/1wXwD8vkfKJz-eB3ivxCAkzLmel2PG-odmHPkXfB8D4Q/edit?usp=sharing

1.مقدمه

نحوه پیاده سازی چند نخی:

از یک استخری از تردها

```
static thread_pool_t g_thread_pool;
```

استفاده می‌کنیم که در آن. تعداد MAX_THREADS ترد ایجاد شده و آماده اند تا بلافاصله یک وظیفه به آنها داده شده و اجرا کنند.(آماده سازی و ایجاد استخر در تابع thread_pool_init)
تمامی این ترد ها تابع worker_thread را اجرا می کنند که برای هر نوع وظیفه شامل:

```
typedef enum {
    TASK_TYPE_UPLOAD_CHUNK,
    TASK_TYPE_DOWNLOAD_CHUNK,
    TASK_TYPE_COMPUTE_HASH,
    TASK_TYPE_WRITE_FILE,
    TASK_TYPE_DELETE_FILE
} task_type_t;
```

دستورات مشخصی دارد، به محض اینکه وظیفه ای به ترد داده شد بسته به مقدار task->type بخش مناسب کد را اجرا می کند و منتظر می ماند تا وظیفه بعدی برسد
برای آپلود یا دانلود با توابع create_upload_chunk_task و create_download_chunk_task یک وظیفه task_t می سازیم و آن را به عنوان ورودی به تابع thread_pool_submit_task می دهیم. این تابع این وظیفه را به یک ترد بیکار می دهد، در صورتی که هیچ تردی بی کار نباشد صبر می کند
وقتی کارمان تمام شد با تابع thread_pool_shutdown استخر را آزاد می کنیم

نحوه پیاده سازی دسترسی به منابع اشتراکی:

منابع مشترک ما فایل های جیسون مانیفست و چانک های داده اند.
دسترسی به منابع مشترک باید با قفل مدیریت شود تا از شرایط رقابتی و data loss جلوگیری شود
برای این منظور به ازای هر در خواستی که از یک چانک داده یا فایل مانیفست استفاده می کند ما یک

```
typedef struct manifest_entry {
    char cid[128];
    manifest_info_t *manifest_info;
    int ref_count;
    time_t last_access;
    pthread_mutex_t lock;
    int marked_for_deletion;
} manifest_entry_t;
```

یا

```
typedef struct chunk_entry {
    char hash_hex[65];
    char *file_path;
    int ref_count;
    pthread_mutex_t lock;
    time_t last_access;
    int marked_for_deletion;
} chunk_entry_t;
```

در آرایه

```
static manifest_entry_t **manifest_table = NULL;
```

یا

```
static chunk_entry_t **chunk_table = NULL;
```

اضافه می کنیم (در صورتی که در حال حاضر وجود نداشته باشد)
و به این نحو کنترل صورت می گیرد که هر چانک یا فایل مانیفستی که در آرایه مربوطه اش ذخیره می‌شود یک قفل دارد که برای خواندن یا نوشتن، باید ترد قفل را بگیرد و بعد از آنکه کارش تمام شد آزاد کند.

علامه بر هر چانک داده یا فایل مانیفست خود آرایه نگه‌دارنده اینها هم منبع اشتراکی است و باید دسترسی به آن مدیریت شود، پس هر آرایه هم یک قفل دارد.
این قفل برای مدیریت دسترسی به یک خانه خاص از آرایه نیست بلکه خود آرایه است (منظور حذف یا اضافه کردن یک عنصر به آرایه)

```
static pthread_mutex_t manifest_table_lock =
PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t chunk_table_lock =
PTHREAD_MUTEX_INITIALIZER;
```

توابع پایه:

تابع ensure_dir

امضای تابع

```
void ensure_dir(const char *path)
```

ورودی: یک آرایه ای از کاراکتر ها به صورت اشاره گر که نشان دهنده زیر پوشه های است که باید به صورت تو در تو ساخته شوند. مثلا /aa/bb

خروجی: ندارد

توابع مهمی که استفاده می کند :

```
snprintf(tmp, sizeof(tmp), "%s", path);
```

snprintf(str, size, format, ...);

یک تابع در کتابخانه **<stdio.h>** است که طول مشخصی از رشته مشخص شده را در بافر می نویسد

str: اشارەگر به بافر

size: حداکثر تعداد بایت(کاراکتر)

format: فرمت رشته که باید نوشته شود

در واقع رشته کنترلی است و در ... آنچه باید به جای مقادیر کنترلی جایگذاری شود می آید

خروجی:

تعداد بایت نوشته شده

اگر خطایی رخ دهد عدد منفی بر می گرداند

```
mkdir(tmp, 0755);
```

tmp : نام دایرکتوری جدیدی که ساخته می‌شود و مکان آن

مثلا اگر a/b/ باشد سعی می کند در پوشه a پوشه b را ایجاد کند و اگر پوشه a وجود نداشته باشد با خطا مواجه می شود

0755 : یک عدد اوکتال (مبنای ۸) که سطح دسترسی را مشخص می‌کند

تفسیر عدد 0755:

رقم	گروه کاربری	مقدار	دسترسی‌ها	توضیح
۰	Special	0	-	بیت‌های خاص (setuid, setgid, sticky)
۷	Owner	7	rwx	مالک: خواندن + نوشتن + اجرا
۵	Group	5	r-x	گروه: خواندن + اجرا
۵	Others	5	r-x	سایرین: خواندن + اجرا

مالک: می‌تواند بخواند، بنویسد و وارد دایرکتوری شود

گروه: فقط می‌تواند بخواند و وارد دایرکتوری شود

سایرین: فقط می‌تواند بخواند و وارد دایرکتوری شود

کار تابع

اگر مسیر به / ختم شود، آن را حذف می‌کند

در یک حلقه از کاراکتر دوم شروع می‌کند (tmp + 1) تا از ایجاد پوشه در ریشه (/) جلوگیری کند

هر زمان به / برسد:

آن را به Nul تبدیل می‌کند که کد اسکی آن 0 است

mkdr پوشه را تا آن نقطه ایجاد می‌کند چون فکر می کند به Null (پایان رشته) رسیده

دوباره / را سرجایش بازمی‌گرداند

تابع send_error

امضای تابع

```
int send_error(int fd, const char *code, const char *message)
```

هدف کلی

ایجاد یک فریم خطا (Error Frame) در قالب JSON و ارسال آن از طریق یک سوکت به کلاینت.

ورودی‌ها

fd: شماره توصیف‌گر فایل (File Descriptor) سوکت که باید خطا روی آن ارسال شود.

code: رشته‌ای (const char) که کد خطا را مشخص می‌کند (مثلاً INVALID_REQUEST یا INTERNAL_ERROR).

message: رشته‌ای (`const char) که توضیح متنی خطا را در بر می‌گیرد.

خروجی

مقدار بازگشتی از تابع send_frame().

توابع مهمی که استفاده می‌کند :

```
snprintf(error_json, sizeof(error_json),
"{\"code\":\"%s\",\"message\":\"%s\"}", code,
message);
```

ساخت یک رشته JSON از کد و پیغام خطا در بافر error_json با طول حداکثر ۲۵۶ بایت.

-اگر رشته‌های code یا message خیلی طولانی باشند، snprintf به صورت خودکار رشته را قطع می‌کند تا در بافر جا شود .

کار تابع

1. ساخت JSON خطا :

- با استفاده از snprintf، یک آبجکت JSON با دو کلید code و message در بافر موقت error_json ساخته می‌شود.

- بافر ۲۵۶ بایتی است، بنابراین طول کل رشته JSON نباید از ۲۵۵ کاراکتر بیشتر شود (به دلیل اینکه آهرین خانه NULL است).

2. ارسال فریم:

- تابع send_frame با پارامترهای زیر فراخوانی می‌شود:

fd: همان شماره سوکت ورودی.

OP_ERROR`: نوع عملیات (کد ثابت برای مشخص کردن نوع فریم خطا).

error_json: محتوای JSON خطا.

strlen(error_json): طول دقیق JSON (نه اندازه کل بافر).

3. بازگشت نتیجه:

- خروجی تابع send_frame مستقیماً به عنوان خروجی این تابع بازگردانده می‌شود.

مثال استفاده:

send_error(client_fd, "AUTH_FAILED", "Invalid username or password");

خروجی تولیدی در بافر:

"code":"AUTH_FAILED","message":"Invalid username or password"

```
void hash_to_hex(const uint8_t *hash, char *hex)
```

هدف

تبدیل یک آرایه بایت (هش) به رشته نمایش هگزادسیمال (مبنای ۱۶) با حروف کوچک.

ورودی‌ها

`hash`: اشاره‌گر به آرایه‌ای از نوع `uint8_t` (بایت بدون علامت) که حاوی داده هش (مثلاً خروجی SHA-256) است.

`hex`: اشاره‌گر به آرایه‌ای از نوع `char` که نتیجه رشته هگزادسیمال در آن ذخیره خواهد شد.

نتیجه نهایی

رشته هگزادسیمال در آرایه `hex` ذخیره می‌شود.

کار تابع

1. پیمایش بایت‌های هش:

- یک حلقه از `i = 0` تا `HASH_LENGTH - 1` اجرا می‌شود.

- مقدار `HASH_LENGTH` باید در جایی از کد (مثلاً هدر فایل) تعریف شده باشد. با توجه به آرایه خروجی (`hex[64]`)، به نظر می‌رسد `HASH_LENGTH = 32` (چون ۳۲ بایت × ۲ کاراکتر = ۶۴ کاراکتر هگز).

2. تبدیل هر بایت:

- برای هر بایت در `hash[i]`:

- تابع `sprintf` دو کاراکتر هگزادسیمال تولید می‌کند (مثلاً بایت `0xAB` به `"ab"` تبدیل می‌شود).

- این دو کاراکتر در محل `hex[i*2]` و `hex[i*2 + 1]` قرار می‌گیرند.

3.پس از پایان حلقه، در index ۶۴ (یعنی `HASH_LENGTH * 2`) کاراکتر `'\0'` (`NULL`) قرار می‌گیرد تا رشته به درستی پایان یابد.

مثال

ورودی :

`{0x1a, 0x3f, 0x8c, ..., 0xff}`

خروجی در `hex`:

`"1a3f8c...ff"` (یک رشته ۶۴ کاراکتری + یک کاراکتر `NULL`).

- خروجی همیشه با حروف کوچک است (به دلیل استفاده از `x%` نه `X%`).

```
int hex_to_hash(const char *hex, uint8_t *hash)
```

هدف

تبدیل یک رشته هگزادسیمال (مبنای ۱۶) به آرایه باینری معادل (هش). این تابع معکوس تابع `hash_to_hex` است.

ورودی‌ها

- `hex`: اشاره‌گر به رشته‌ای (`const char*`) که نمایش هگزادسیمال هش را دارد. باید فقط شامل کاراکترهای مجاز هگز (۰-۹، a-f، A-F) باشد و طول آن حداقل `HASH_LENGTH * 2` کاراکتر باشد.
- `hash`: اشاره‌گر به آرایه‌ای از نوع `uint8_t` که نتیجه باینری هش در آن ذخیره خواهد شد.

خروجی

مقدار بازگشتی از نوع `int`:
- `0`: در صورت موفقیت‌آمیز بودن تبدیل تمام بایت‌ها.
- `-1`: در صورت بروز خطا (وجود کاراکتر غیرمجاز در رشته ورودی یا فرمت نامعتبر).

توابع مهمی که استفاده می‌کند

```
sscanf(hex + i * 2, "%2hhx", &hash[i])
```

این تابع دو کاراکتر هگزادسیمال از موقعیت `hex + i * 2` را خوانده و به مقدار باینری تبدیل می‌کند.
`hex + i * 2`: آدرس شروع دو کاراکتر هگز برای بایت `i`ام.
`"%2hhx"`: فرمت‌دهنده که دقیقاً دو کاراکتر هگز را خوانده و به یک بایت بدون علامت (`unsigned char`) تبدیل می‌کند.

`&hash[i]`: آدرس محل ذخیره نتیجه برای بایت `i`ام.

کار تابع

1. پیمایش بایت‌ها:
- یک حلقه از `i = 0` تا `HASH_LENGTH - 1` اجرا می‌شود.
- مقدار `HASH_LENGTH` باید در جایی از کد تعریف شده باشد (احتمالاً ۳۲ برای SHA-256).

2. خواندن و تبدیل هر جفت کاراکتر:
- در هر تکرار، دو کاراکتر از رشته `hex` (شروع از موقعیت `i*2`) توسط `sscanf` خوانده و به یک بایت تبدیل می‌شود.
- اگر `sscanf` نتواند یک مقدار معتبر بخواند (یعنی مقدار بازگشتی آن برابر ۱ نباشد)، تابع بلافاصله با خطا (`-1`) بازمی‌گردد.

3. بازگشت نتیجه:
- اگر تمام ۳۲ بایت با موفقیت تبدیل شدند، تابع `0` را برمی‌گرداند.

مثال

ورودی (`hex`): `"1a3f8c...ff"` (یک رشته ۶۴ کاراکتری)
خروجی در `hash`: آرایه‌ای ۳۲ بایتی به صورت `{0x1a, 0x3f, 0x8c, ..., 0xff}`

```
void compute_hash(const uint8_t *data, size_t
len, uint8_t *hash)
```

در متن پروژه پیشنهاد شده از Blake3 استفاده کنیم و قابل تغییر با متغیر HASH_ALGO باشد اما در کامپایل کردن با استفاده از کتابخانه های Blake3 لینکر ارور می‌داد در نتیجه از sha استفاده کردیم.
هدف کلی
محاسبه هش SHA-256 از داده ورودی و ذخیره نتیجه در آرایه خروجی.

ورودی‌ها

- `dat`: اشاره‌گر به آرایه‌ای از uint8_t (بایت بدون علامت) که حاوی داده ورودی برای محاسبه هش است.
- `len`: طول داده ورودی بر حسب بایت (از نوع `size_t`).
- `hash`: اشاره‌گر به آرایه‌ای از `uint8_t` که نتیجه هش ۳۲) SHA-256 بایت) در آن ذخیره خواهد شد.

این تابع از کتابخانه OpenSSL استفاده می‌کند. برای کامپایل باید -lcrypto به لینکر اضافه شود.

```
#include <openssl/sha.h>
```

آرایه `hash` باید حداقل ۳۲ بایت فضا داشته باشد.

```
int store_manifest(const char *cid, const char
*manifest_json)
```

فایل های مانیفست منابعی است که اند که ترد ها در دانلود و آپلود ممکن است نیاز به خواندن و نوشتن از آنها داشته باشند، برای اینکه شرایط رقابتی race condition پیش نیاید یعنی نتیجه نهایی به ترتیب دسترسی ترد ها به فایل بستگی نداشته باشد باید نوشتن به شکل یک دستور اتمیک انجام شود (میانه‌ی اجرای دستورات اتمیک CPU به فرایند دیگری داده نمی شود، به بیان ساده ممکن نیست دستورات اتمیک نیمه‌کاره رها شوند)
اما دستوری برای نوشتن اتمیک در یک فایل نداریم
پس از یک روش جالب استفاده می کنیم:
متن مانیفست را در یک فایل جیسون در همان مکانی که می خواهیم فایل اصلی باشد به طور موقت می نویسیم، سپس طی یک دستور اتمیک نام فایل را به نام اصلی (همان cid) تغییر می دهیم

2.ساختار پروژه

2.1. اپلود

سه تابع اصلی دارد:

```
upload_state_t* handle_upload_start(int client_fd, const uint8_t
*payload, uint32_t len)
```

ساختار upload_state_t را برای یک در خواست آپلود می سازد
این ساختار تمام اطلاعات لازم مربوط به آپلود را در بر دارد

```
void handle_upload_chunk(int client_fd, const uint8_t *payload,
uint32_t len,upload_state_t *state)
```

وظیفه آپلود را ایجاد و به یک نخ در استخر ترد ها می دهد

```
void handle_upload_finish(int client_fd, upload_state_t *state)
```

ساختار upload_state_t را آزاد می کند

2.2. دانلود

توضیح توابع مربوطه اضافه شده

2.2.1. Read_manifest_file:

کل فایل را میخواند و بصورت یک رشته برمیگرداند .

خلاصه جریان کاری:

1. ساخت مسیر ← مسیر فایل مانیفست بر اساس CID ساخته می‌شود
2. باز کردن فایل ← فایل برای خواندن باز می‌شود
3. اندازه‌گیری ← اندازه فایل تعیین می‌شود(با رفتن به انتهای فایل و اندازه گیری موقعیت پوینتر)
4. اعتبارسنجی اندازه ← اطمینان از اینکه فایل خیلی بزرگ نیست
5. تخصیص حافظه ← حافظه کافی برای محتوای فایل اختصاص می‌یابد
6. خواندن داده ← تمام محتوای فایل به حافظه خوانده می‌شود

7. تبدیل به رشته ← داده به رشته C معتبر تبدیل می‌شود
8. بستن و بازگشت ← فایل بسته شده و داده برگردانده می‌شود

2.2.2. Parse_manifest:

ذخیره اطلاعات رشته ورودی در استراکت manifest_info

خلاصه جریان کاری:

1. استخراج metadata:

- نام فایل (رشته)
- اندازه کل (عدد)

2. شمارش چانک‌ها:

- با شمارش وقوع "index"

شمارش تعداد چانک‌ها:

- **الگوریتم**:
  1. جستجوی `"index"` در رشته
  2. اگر پیدا شد:
     - افزایش شمارنده
     - افزایش اشاره‌گر برای ادامه جستجو از کاراکتر بعدی
  3. تکرار تا انتهای رشته
- **منطق**: هر چانک باید یک فیلد `index` داشته باشد

3. تخصیص حافظه:

- برای آرایه چانک‌ها

4. استخراج اطلاعات هر چانک:

- شماره چانک (index)
- اندازه چانک (size)
- هش چانک (hash) + تبدیل هگز به باینری

تبدیل هگزادسیمال به باینری:(برای ذخیره در مانیفست)

حلقه: برای هر بایت هش (فرضاً HASH_LENGTH = 32)

تابع: sscanf - خواندن فرمت‌دار از رشته

فرمت: "2hhx%" - خواندن دو کاراکتر هگز به عنوان یک بایت

محاسبه آدرس: hash_hex + j * 2 - جابجایی 2 کاراکتر در هر مرحله

ذخیره: در آرایه hash_code چانک

- ساخت مسیر فایل از هش

5. ساختاردهی داده‌ها:

- پر کردن ساختار manifest_info

2.2.3. Cleanup_manifest_info

فقط در صورتی که حافظه‌ای تخصیص داده شده باشد، آن را آزاد می‌کنیم

فقط اگر آرایه چانک‌ها تخصیص داده شده باشد، آن را پردازش می‌کنیم

مقدار دهی اشاره گر ها به null -> اگر بعداً دوباره بررسی کنیم، می‌دانیم که این حافظه آزاد شده است

مقدار دهی متغیر ها به صفر

2.2.4. Verify_chunk

این تابع مسیر فایل چانک و هش مورد انتظار را دریافت می‌کند و بررسی می‌کند که هش محاسبه شده از فایل با هش مورد انتظار مطابقت دارد یا خیر

جریان کاری تابع:

1. باز کردن فایل:

اگر خطا ← بازگشت -1

2. اندازه‌گیری فایل:

fseek(به انتها)

ftell(گرفتن اندازه)

fseek(به ابتدا)

3. خواندن فایل:

malloc(اندازه فایل)

↓

اگر خطا ← بستن فایل و بازگشت -1

fread(کل فایل)

fclose(فایل)

4. محاسبه هش:

calculate_hash(داده فایل, اندازه, خروجی هش)

5. پاکسازی و مقایسه

تغییرات در `handle_connection`

2.2.5. (op == OP_DOWNLOAD_START)

توضیح برخی خط های نا مفهوم:

```
memset(&manifest, 0, sizeof(manifest));
```

صفر کردن: `memset` تمام بایت‌های ساختار را 0 می‌کند

هدف: جلوگیری از داده‌های تصادفی و مقداردهی پیش‌فرض

```
usleep(1000); // 1ms
```

تاخیر بین ارسال چانک‌ها

- **تابع**: `usleep` - توقف اجرا به میکروثانیه
- **مدت**: 1000 میکروثانیه = 1 میلی‌ثانیه
- **هدف**: جلوگیری از overload شبکه/سیستم

خلاصه جریان کاری:

فاز 1: آماده‌سازی

1. دریافت CID از کلاینت
2. خواندن فایل مانیفست
3. تجزیه JSON و پر کردن ساختار
4. نمایش اطلاعات فایل

فاز 2: ارسال چانک‌ها

برای هر چانک:

1. تأیید اعتبار چانک (verify_chunk)

2. باز کردن فایل چانک

3. خواندن داده به حافظه

4. ساخت payload با هدر

5. ارسال به کلاینت

6. پاکسازی موقت

7. تاخیر کوتاه

فاز 3: پایان‌دهی

1. ارسال سیگنال پایان
2. پاکسازی تمام منابع

نکات مهم طراحی:

1. مدیریت خطا:

- هر چانک به طور مستقل پردازش می‌شود
- خطا در یک چانک مانع ارسال بقیه نمی‌شود
- از `continue` برای رد کردن چانک معیوب استفاده می‌شود

2. مدیریت حافظه:

- هر چانک حافظه خود را تخصیص و آزاد می‌کند
- پاکسازی کامل در پایان انجام می‌شود
- بررسی NULL بعد از هر `malloc`

3. کارایی:

- تاخیر 1ms بین چانک‌ها برای جلوگیری از overload
- خواندن کل فایل در حافظه (مناسب برای چانک‌های کوچک)

4. قابل اطمینان بودن:

- راستی‌آزمایی هر چانک قبل از ارسال
- استفاده از network byte order برای سازگاری
- لاگ‌گیری کامل از مراحل

2.3. دانلود همزمان

قفل ها:

```
;pthread_mutex_t manifest_mutex = PTHREAD_MUTEX_INITIALIZER
;pthread_rwlock_t blocks_rwlock = PTHREAD_RWLOCK_INITIALIZER
;pthread_mutex_t refcount_mutex = PTHREAD_MUTEX_INITIALIZER
```

در پروژه ما که از معماری چندنخی (multi-threaded) استفاده می‌کند، مدیریت دسترسی همزمان به منابع اشتراکی حیاتی است. ما از سه نوع قفل مختلف برای اطمینان از ایمنی رشته‌ها (thread safety) و جلوگیری از شرایط رقابت (race conditions) استفاده کرده‌ایم.

1.قفل مانیفست:

نوع قفل

- **Mutex ساده** (انحصاری متقابل)
- **Initializer**: PTHREAD_MUTEX_INITIALIZER - مقداردهی اولیه استاتیک

هدف و کاربرد

این قفل دسترسی به **فایل‌های مانیفست** را مدیریت می‌کند. مانیفست‌ها فایل‌های متادیتایی هستند که اطلاعات فایل‌های ذخیره شده (مانند نام فایل، تعداد چانک‌ها، هش چانک‌ها و غیره) را نگهداری می‌کنند.

دلیل انتخاب Mutex ساده

1. **عملیات‌های کم تکرار**: خواندن و نوشتن مانیفست نسبتاً کم اتفاق می‌افتد
2. **عملیات‌های سریع**: خواندن/نوشتن مانیفست عملیات‌های کوتاهی هستند
3. **سادگی**: پیاده‌سازی ساده‌تر و overhead کمتر نسبت به read-write lock

2. قفل بلاک ها

نوع قفل

- **Read-Write Lock** (قفل خواننده-نویسنده)
- **Initializer**: PTHREAD_RWLOCK_INITIALIZER - مقداردهی اولیه استاتیک

هدف و کاربرد

این قفل دسترسی به **فایل‌های چانک/بلاک** را مدیریت می‌کند. بلاک‌ها واحدهای ذخیره‌سازی داده در سیستم ما هستند

دلیل انتخاب Read-Write Lock

1. **الگوی دسترسی نامتقارن**:
   - **خواندن زیاد**: ده‌ها/صدها دانلود همزمان
   - **نوشتن کم**: فقط هنگام آپلود فایل جدید
2. **کارایی بهینه**:
   - چندین thread می‌توانند همزمان **بخوانند**
   - فقط یک thread می‌تواند **بنویسد**
   - خواننده‌ها مانع یکدیگر نمی‌شوند

3. **جلوگیری از گلوگاه**:
   - در صورتی که فقط از mutex استفاده می‌کردیم، همه دانلودها به صورت سریال انجام می‌شدند
   - با read-write lock، دانلودهای همزمان می‌توانند موازی اجرا شوند

3.قفل شمارنده ارجاع:

نوع قفل

- **Mutex ساده**
- **Initializer**: `PTHREAD_MUTEX_INITIALIZER`

هدف و کاربرد

این قفل دسترسی به **ساختار داده شمارنده ارجاع (reference counting)** را مدیریت می‌کند. این ساختار تعداد دانلودهای فعال از هر بلاک را پیگیری می‌کند

دلیل انتخاب Mutex ساده

1. **عملیات‌های بسیار کوتاه**: افزایش/کاهش یک عدد صحیح
2. **ساختار داده کوچک**: لیست پیوندی با تعداد محدود گره
3. **کمترین overhead**: برای عملیات‌های اتمیک کوتاه مناسب است

مقایسه کلی قفل‌ها

قفل	نوع	الگوی دسترسی	دفعات استفاده	Overhead
`manifest_mutex`	Mutex ساده	خواندن/نوشتن کم	کم	پایین
`blocks_rwlock`	Read-Write Lock	خواندن زیاد، نوشتن کم	بسیار زیاد	متوسط
`refcount_mutex`	Mutex ساده	خواندن/نوشتن متوسط	متوسط	پایین

استراتژی قفل‌گذاری در معماری سیستم

1. سطح دانه‌بندی (Granularity) مناسب:

- قفل جداگانه برای مانیفست‌ها
- قفل جداگانه برای بلاک‌ها
- قفل جداگانه برای reference counting) metadata)

2. جلوگیری از Deadlock:

- **الگوی ثابت**: همیشه قفل‌ها را به همان ترتیب می‌گیریم
- **رهایی تضمینی**: همیشه در مسیرهای خروج، قفل آزاد می‌شود

3. تعادل بین کارایی و ایمنی:

- **Mutex**: برای منابع با رقابت کم
- **RWLock**: برای منابع با خواندن زیاد
- **Initializer استاتیک**: ساده و بدون نیاز به تابع init جداگانه

توضیح توابع مربوطه اضافه شده:

2.3.1. increment_ref_count:

یک رشته حاوی هش هگزادسیمال بلاک را دریافت می‌کند و تعداد ارجاع جدید را برمی‌گرداند.

```
pthread_mutex_lock(&refcount_mutex);//lock
```

قفل کردن mutex برای اطمینان از دسترسی thread-safe به ساختارهای داده اشتراکی. این خط تضمین می‌کند که فقط یک thread در هر لحظه می‌تواند به لیست block_refs دسترسی داشته باشد

```
        struct block_ref *current = block_refs;//اشاره گر به اول لیست پیوندی
```

لیست تمام بلاک‌ها و تعداد ارجاع‌هایشان را نگهداری می‌کند

2.3.2. decrement_ref_count:

```
struct block_ref **prev = &block_refs;//double pointer
```

ایجاد یک اشارەگر دوگانه (double pointer) است که به آدرس اشارەگر سرلیست (`block_refs`) اشاره می‌کند. این برای حذف گره از لیست لازم است

```
*prev = current->next;//delete node
```

حذف گره از لیست پیوندی. `*prev` همان اشارەگر قبلی است که به `current` اشاره می‌کرد. حالا آن را به گره بعدی `current->next` تنظیم می‌کنیم، بنابراین گره `current` از لیست حذف می‌شود

نکات کلیدی دو تابع 2.3.1 و 2.3.2

این توابع هسته اصلی مدیریت همزمانی در پروژه هستند و تضمین می‌کنند که بلاک‌های در حال استفاده حذف نمی‌شوند و داده‌ها در محیط چندنخی ایمن باقی می‌مانند.

1. **Reference Counting**: این مکانیزم برای ردیابی تعداد دانلودهای فعال از هر بلاک استفاده می‌شود تا از حذف تصادفی بلاک‌هایی که هنوز در حال استفاده هستند جلوگیری شود.
2. **Thread Safety**: استفاده از mutex تضمین می‌کند که در محیط چندنخی، فقط یک thread در هر لحظه بتواند لیست را تغییر دهد.
3. **لیست پیوندی**: داده‌ها در یک لیست پیوندی ساده ذخیره می‌شوند که هر گره شامل:
   - هش بلاک
   - تعداد ارجاع‌ها
   - اشاره‌گر به گره بعدی
4. **درج در ابتدا**: در تابع `increment_ref_count`، گره جدید همیشه در ابتدای لیست درج می‌شود (درج سر). این کار O(1) است.
5. **حذف از وسط**: در تابع `decrement_ref_count`، اگر نیاز به حذف گره باشد، از الگوی "حذف از لیست پیوندی با دنبال کردن اشاره‌گر قبلی" استفاده می‌شود.
6. **مدیریت حافظه**: در صورت کاهش شمارنده به صفر، حافظه گره بلافاصله آزاد می‌شود.
7. **خطاها**: توابع در صورت خطا (مثل عدم موفقیت در تخصیص حافظه یا پیدا نکردن بلاک) مقادیر منفی برمی‌گردانند.

2.3.3. read_chunk_file_safe:

`file_size`: اشاره‌گری به متغیر `long` که اندازه فایل در آن ذخیره می‌شود را دریافت می‌کند و یک اشاره‌گر به داده‌های خوانده شده (آرایه بایت) برمی‌گرداند

اهمیت استفاده از Read-Write Lock :

1. **همزمانی خواندن**: چندین thread می‌توانند همزمان فایل‌های چانک را بخوانند
2. **جلوگیری از تداخل**: در حین خواندن چانک، هیچ thread دیگری نمی‌تواند همان چانک را بنویسد یا تغییر دهد
3. **کارایی**: چون دانلودهای همزمان ممکن است چانک‌های مشترکی را بخوانند، این قفل اجازه می‌دهد همه آن‌ها به طور موازی کار کنند

2.3.4. handle_download:

این تابع هسته اصلی عملیات دانلود در سیستم ذخیره‌سازی توزیع شده است. نسخه بهبود یافته آن از قفل‌گذاری و reference counting برای پشتیبانی از دانلودهای همزمان استفاده می‌کند

جریان کاری کلی:

1. خواندن مانیفست (با قفل)

2. تجزیه مانیفست

3. افزایش reference count تمام چانک‌ها

4. برای هر چانک:

a. راستی‌آزمایی (با قفل)

b. خواندن داده (با قفل)

c. ساخت payload

d. ارسال به کلاینت

e. آزادسازی منابع

5. علامت پایان دانلود

6. کاهش reference count

7. پاکسازی